

EngineeringOS — الخطة التنفيذية الكاملة لوكيل الذكاء

الاصطناعي

النسخة: Execution Directive v1.0

الغاية: الوصول إلى رؤية Engineering Digital Twin / Engineering Intelligence Platform بأقل التعديلات الممكنة على البنية الحالية.

(1) المهمة العليا

على وكيل الذكاء الاصطناعي أن يتعامل مع EngineeringOS باعتباره منصة موجودة بالفعل وليست مشروعًا يبدأ من الصفر. البنية الحالية تحتوي على طبقة عقود contract-first، وطبقة بيانات فعلية، ومحرك تنفيذ، واكتشاف onboarding، و knowledge graph، ومؤشرات observability/traceability، لذلك المطلوب ليس إعادة التصميم، بل تثبيت ما هو قائم وإضافة الطبقات الناقصة فقط.

(2) قواعد العمل غير القابلة للكسر

1. لا تلغ أي طبقة موجودة تعمل جيدًا.
OpenAPI هو المصدر المرجعي، وتوليد Zod / React Query hooks يجب أن يبقى تابعًا للعقد، لا العكس.
2. لا تدخل إعادة بناء شاملة.
أقل تعديل هو المبدأ. المطلوب إضافة timeline، snapshots، موحد، provenance/trust، أوضح، RBAC، simulation API، و advisory layer خفيفة فوق اللبنة الحالية.
3. لا تخرج أي استنتاج بلا دليل.
كل inference يجب أن يعود إلى evidence/source/provenance ودرجة ثقة مناسبة؛ هذا أساسي لبناء التوأم الرقمي.
4. كل تغيير يجب أن يمر عبر اختبارات أو فحوصات اتساق.
لدى المشروع اختبارات في scanner وال API server وال job queue وال plugin runtime، ويجب ألا يكسر الوكيل ما هو مغطى أصلاً.

(3) الصورة الحالية التي يجب البناء فوقها

المشروع يحتوي فعليًا على:

- pipeline عقدي contract-first: OpenAPI → Zod → React Query → API server.
- قاعدة بيانات تحتوي المشاريع والمهام والقواعد والسير والأحداث والقياسات و graph والتدقيق والاكتشاف و scan jobs وسجلات المهام وال plugins.
- execution engine مع queue محدود و reconciliation و scan runner و transactional writes.
- discovery onboarding يبدأ بالاكتشاف بدل form تقليدي.
- knowledge graph / inference layer لإيجاد impact path clusters summary.
- observability عبر events و task_logs و audit_logs و metrics و correlationId.

وفي المقابل، ما يزال ناقصًا أو جزئيًا:

- RBAC أو project-scoped permissions.
- simulation مستقلة من نوع "ماذا لو؟".
- timeline موحد يجمع scan / events / audit / tasks / workflows / metrics.
- plugin platform معزول أو أوسع من in-process runtime.
- advisory engine صريح يخرج توصيات هندسية قابلة للتنفيذ.

(4) تعريف المخرجات المطلوبة من الوكيل

على الوكيل أن يسلم النظام في النهاية إلى حالة يمكن فيها:

1. استيراد أي مشروع، ثم بناء Knowledge Snapshot له.
2. عرض Timeline موحد لتطور المشروع.
3. توضيح provenance + confidence لكل استنتاج في graph والواجهة وAPI.
4. تشغيل Simulation API تعطى impact/risk/broken contracts/estimated fallout.
5. فرض RBAC / project-scoped access على العمليات الحساسة.
6. تقديم Advisory Layer تخبر بما يجب تغييره بأقل كلفة وأعلى أثر.

(5) ترتيب التنفيذ الأدنى

المرحلة 0 — تثبيت الحقيقة المرجعية

الهدف: عدم العمل على افتراضات قديمة.

ما يجب فعله

- اعتماد docs/completion-plan.md و docs/fact-record.md كمرجعية تشغيلية.
- مراجعة docs و agents/memory قبل أي تعديل حتى لا يُكسر سلوك متراكم أو قرار سابق.
- اعتبار lib/api-spec/openapi.yaml المصدر الأساسي لكل endpoint أو schema.

المخرجات

- قائمة بالحقيقة الحالية لكل طبقة.
- نقاط لا يجوز تغييرها دون سبب.
- قائمة الملفات التي يجب ألا تُعاد كتابتها إلا إذا اقتضت الحاجة.

المرحلة 1 — Knowledge Snapshot / Versioning

الهدف: تحويل كل scan/discovery إلى لقطة معرفة قابلة للتخزين والمقارنة.

المطلوب

أضف طبقة تحفظ بعد كل عملية scan أو discovery:

- entities
- relationships
- metrics
- provenance
- correlationId
- version
- timestamp

أين يبدأ التعديل

- lib/db/src/schema/graph.ts
- lib/db/src/schema/metrics.ts
- lib/db/src/schema/audit_logs.ts
- artifacts/api-server/src/lib/scan-runner.ts
- artifacts/api-server/src/routes/discovery.ts
- artifacts/api-server/src/routes/projects.ts

سلوك التنفيذ

1. عند اكتمال scan/discovery، أنشئ snapshot جديدًا.
2. اربط snapshot بالمشروع وبالجلسة وبال scan job إن وُجد.
3. اجعل snapshot يحتوي حالة المعرفة كاملة وليس metrics فقط.
4. وقر مقارنة بين snapshot الحالي والسابق على الأقل في backend.

معايير القبول

- يمكن استرجاع آخر snapshot لكل مشروع.
- يمكن مقارنة snapshotين متتاليين.
- لا يضع provenance أثناء الحفظ.
- لا يتعارض هذا مع ال transactional writes الحالية.

المرحلة 2 — Timeline موحد

الهدف: جعل المشروع يمتلك خطًا زمنيًا واحدًا يقرأ تطوره الحقيقي.

المطلوب

إنشاء endpoint / خدمة تجمع:

- scans
- audit logs
- events

- task logs •
- workflow transitions •
- metrics snapshots •

أين يبدأ التعديل

- artifacts/api-server/src/routes/events.ts •
- artifacts/api-server/src/routes/dashboard.ts •
- artifacts/api-server/src/routes/projects.ts •
- artifacts/dashboard/src/pages/ProjectDetail.tsx •
- artifacts/dashboard/src/pages/Dashboard.tsx •

سلوك التنفيذ

1. أنشئ view أو query layer موحداً للـ timeline.
2. رتب الأحداث زمنياً مع projectId و entityId و actor و source type.
3. اربط كل timeline item بالدليل المتاح له.
4. اعرض timeline في الواجهة كمسار زمني قابل للتصفية.

معايير القبول

- يمكن للمستخدم رؤية تاريخ المشروع في شاشة واحدة.
- يمكن فلترة الخط الزمني حسب النوع أو المصدر أو الفترة.
- لا تتكرر الأحداث المتماثلة بلا سبب.
- يظل كل حدث قابلاً للتتبع إلى مصدره الأصلي.

Provenance + Trust — المرحلة 3

الهدف: منع الاستنتاجات الغامضة وتحويل graph من "بيانات" إلى "معرفة موثوقة".

المطلوب

اجعل لكل node/edge أو inference:

- source •
- evidence •
- confidence •
- lastVerified •
- extraction method •
- impact weight •

أين يبدأ التعديل

- lib/db/src/schema/graph.ts •
- lib/knowledge-engine/src/* •
- artifacts/api-server/src/routes/graph.ts •

artifacts/dashboard/src/pages/Graph.tsx •

سلوك التنفيذ

1. لا تعرض inference بلا evidence واضح.
2. يبين confidence بجانب كل استنتاج.
3. ميّز بين الحقيقة المأخوذة مباشرة والاستنتاج الناتج عنها.
4. وقّر provenance detail view في الواجهة.

معايير القبول

- كل relation أو impact result له مصدر.
- المستخدم يستطيع معرفة لماذا استنتج هذا الرابط.
- يمكن تمييز low-confidence results بصريًا.
- لا يحصل التباس بين raw data و derived knowledge.

المرحلة 4 — Simulation API

الهدف: تمكين "ماذا لو؟" على مستوى المشروع.

المطلوب

محاكاة تغييرات مثل:

- حذف ملف
- تعديل endpoint
- تغيير schema
- نقل module
- تعديل workflow

الناتج المطلوب

- impacted entities
- broken contracts
- risk score
- estimated fallout
- affected tests
- rollback complexity

أين يبدأ التعديل

- lib/knowledge-engine/src/queries.ts •
- lib/knowledge-engine/src/inference.ts •
- artifacts/api-server/src/routes/graph.ts • أو route جديد
- artifacts/dashboard/src/pages/Graph.tsx •

سلوك التنفيذ

1. استقبل "scenario input" واضحًا.
2. شغل inference على knowledge graph الحالي.
3. استخرج قائمة التأثيرات المباشرة وغير المباشرة.
4. احسب risk score موحد.
5. اعرض نتائج المحاكاة قبل أي تنفيذ فعلي.

معايير القبول

- يمكن تشغيل المحاكاة دون لمس الكود الحقيقي.
- يمكن تفسير نتائج المحاكاة.
- النتائج تتضمن dependency impact وليس مجرد file list.
- يمكن حفظ سيناريو المحاكاة أو مقارنته لاحقًا.

المرحلة 5 — RBAC / Project-Scoped Access

الهدف: إغلاق أكبر فجوة حوكمة في النظام.

المطلوب

إضافة صلاحيات على الأقل على مستوى:

- admin
- editor
- viewer
- agent أو على مستوى المشروع والعضوية.

أين يبدأ التعديل

- `artifacts/api-server/src/middlewares/requireAuth.ts`
- `artifacts/api-server/src/app.ts`
- `lib/db/src/schema/*` عند الحاجة إلى `memberships/roles`

سلوك التنفيذ

1. اجعل القراءة/الكتابة/التنفيذ/الاستيراد/الحذف خاضعة لسياسة واضحة.
2. اجعل الصلاحيات project-scoped حيثما أمكن.
3. امنع الوصول الشامل الافتراضي للمستخدم المصادق فقط.
4. اربط الأدوار بالسياسات التنفيذية في API server.

معايير القبول

- لا يستطيع أي مستخدم مصادق الوصول لكل شيء افتراضيًا.
- كل route حساس له policy واضح.
- agent access يمكن عزله عن human access.

• لا تكسر تجربة الـ discovery أو الـ dashboard الأساسية.

المرحلة 6 — Advisory Engine خفيف

الهدف: تحويل نتائج graph والتحليل إلى توصيات عملية.

المطلوب

توليد توصيات مثل:

- أكثر الملفات حساسية
- أقل تعديل لتحقيق الهدف
- أولويات الإصلاح
- مخاطر التغيير
- مناطق الديون التقنية الأعلى
- ترتيب التنفيذ المقترح

أين يبدأ التعديل

```
lib/knowledge-engine/src/inference.ts •
artifacts/api-server/src/routes/graph.ts •
artifacts/dashboard/src/pages/ProjectDetail.tsx •
```

سلوك التنفيذ

1. استخرج signal من metrics/graph/timeline/snapshots.
2. حوّل signal إلى recommendation objects واضحة.
3. اربط كل توصية بالمصدر والسبب والثقة.
4. اعرض التوصيات في Project Detail و Graph.

معايير القبول

- التوصية قابلة للتفسير.
- التوصية لا تظهر كحكم سحري غير مبرر.
- التوصيات تعتمد على بيانات المشروع نفسه.
- يمكن أن تكون "منخفضة التدخل" فعلاً لا شعارات عامة.

6) ما يجب أن يتركه الوكيل كما هو

لا يعاد بناء هذه الأجزاء إلا إذا ظهرت ضرورة حقيقية:

```
lib/api-spec/openapi.yaml •
lib/api-zod/src/generated/* •
```

- lib/api-client-react/src/generated/*
- lib/scanner/src/file-walker.ts
- lib/scanner/src/rule-matcher.ts
- artifacts/api-server/src/lib/job-queue.ts
- artifacts/api-server/src/lib/job-reconciliation.ts
- نواة tasks.ts و workflows.ts الحالية

هذه الأجزاء تمثل البنية المستقرة التي يجب أن تُستثمر لا أن تُهدم.

(7) ترتيب الملفات حسب الأولوية العملية

أولوية قصوى

- lib/db/src/schema/graph.ts .1
- artifacts/api-server/src/lib/scan-runner.ts .2
- lib/knowledge-engine/src/inference.ts .3
- lib/knowledge-engine/src/queries.ts .4
- artifacts/api-server/src/routes/graph.ts .5
- artifacts/api-server/src/middlewares/requireAuth.ts .6

أولوية ثانية

- artifacts/api-server/src/routes/discovery.ts .1
- artifacts/api-server/src/routes/projects.ts .2
- artifacts/api-server/src/routes/events.ts .3
- artifacts/api-server/src/routes/dashboard.ts .4
- artifacts/dashboard/src/pages/Graph.tsx .5
- artifacts/dashboard/src/pages/ProjectDetail.tsx .6
- artifacts/dashboard/src/pages/Dashboard.tsx .7

أولوية ثالثة

- lib/db/src/schema/metrics.ts .1
- lib/db/src/schema/audit_logs.ts .2
- lib/db/src/schema/events.ts .3
- lib/db/src/schema/task_logs.ts .4
- أي ملفات schema إضافية تتطلب دعم roles/memberships .5

(8) أسلوب التنفيذ داخل الوكيل

على الوكيل أن يعمل بهذه الدورة لكل مرحلة:

1. اقرأ الحقيقة الحالية من docs + memory + files المعنية.
2. حدد أقل عدد ملفات يحتاج تعديلًا.
3. أنشئ أو عدّل schema فقط عند الحاجة.
4. حدّث API contract إذا لزم.
5. عدّل التنفيذ الخلفي.
6. عدّل الواجهة فقط بعد ثبوت الحقيقة في backend.
7. أضف/عدّل الاختبارات.
8. شغل فحوص الاتساق.
9. اكتب summary واضحًا: ما تغير، ما لم يتغير، وما بقي.

(9) معايير النجاح النهائية

تكون المهمة ناجحة عندما يتحقق الآتي:

- أي مشروع جديد يدخل عبر discovery ثم يتحول إلى knowledge snapshot موحد.
- الخط الزمني للمشروع يصبح واضحًا ومشاركًا بين scan/event/audit/task/workflow.
- كل استنتاج في graph مرفق بدليل وثقة وتاريخ تحقق.
- يمكن محاكاة أي تغيير رئيسي قبل تنفيذه.
- الصلاحيات لم تعد عامة بلا حدود.
- التوصيات التي يخرجها النظام مبنية على بيانات قابلة للتفسير.
- لم يتم هدم البنية الحالية، بل تم توسيعها بأقل تعديلات ممكنة.

(10) مخرجات التسليم المطلوبة من الوكيل

1. تحديثات الكود اللازمة فقط.
2. سجل حقيقة محدث.
3. خطة استكمال محدثة.
4. اختبارات أو فحوص تثبت عدم كسر النواة.
5. ملخص تنفيذي يوضح:

- ما أنجز
- ما بقي
- ما تغير في الرؤية بعد التنفيذ

الخاتمة

المطلوب من EngineeringOS ليس أن يصبح "برنامجاً أكبر"، بل أن يصبح طبقة معرفة هندسية حيّة. ولهذا السبب يجب أن يتحرك الوكيل من الداخل إلى الخارج:

snapshot → timeline → provenance → simulation → RBAC → advisory

هذا هو أقصر مسار عملي يحقق الرؤية دون تشويش البنية الحالية.